

A Systematic Framework for Backtesting and Live Paper Trading of Quantitative Cryptocurrency Strategies

Author: Saksham

Affiliation: B.Tech Final Year (Capstone Project)

Date: September 2026

Abstract

Cryptocurrency markets trade 24/7 with high volatility, fat-tailed returns and material transaction costs, making the design and *honest* evaluation of systematic strategies difficult. This paper presents an end-to-end Python framework that unifies data acquisition, indicator engineering, strategy implementation, vectorised and event-driven backtesting, risk/execution modelling, performance analytics and a real-time paper/live engine. The strategy set spans a standard *intermediate & advanced* crypto strategy curriculum: Ichimoku Cloud, calendar anomalies, Aroon/RSI divergence, time-series and cross-sectional momentum, cointegrated pairs trading, K-Means market clustering and Hurst-exponent regime filtering.

The framework is validated with 53 unit tests that enforce causality (no look-ahead) and demonstrated on real Binance data (BTC/USDT 1h, 4,319 bars, 180 days). With zero-cost assumptions removed and 0.1% per-side fees plus slippage applied, every single-symbol strategy underperforms buy-and-hold in the sample, which we report transparently; the K-Means-clustered, long-only momentum portfolio also underperformed a BTC buy-and-hold (8,083.26 vs 9,574.27, -15.6%) while still beating its two weakest members (ADA, XRP). A paper/live engine ran overnight on BTC/USDT 1m and traded through an order-book-walking simulator with ATR stops; the full journal (paper + replay + live) records 57 round-trip trades and 116 fills. We argue the framework's contribution is validated *plumbing* — realistic costs, risk and causality — rather than a proven profitable signal, and outline the path to forward-testing ML-driven signals.

Keywords: algorithmic trading, cryptocurrency, Ichimoku, pairs trading, K-Means, Hurst exponent, event-driven backtesting, transaction costs.

1. Introduction

Retail and even institutional access to crypto exchange data is now trivial; what remains hard is turning that data into a *defensible* trading system. Three problems dominate:

1. **Look-ahead bias** — naive strategies inadvertently use future information, inflating backtest performance.
2. **Ignored costs** — ignoring fees, spread, slippage and market impact turns a "profitable" backtest into a losing account.
3. **Unrealistic fill modelling** — assuming fills at the signal bar's close rather than the ground truth of next-bar execution.

This project builds a framework in which these three problems are first-class citizens, and uses it to implement and evaluate a full curriculum of systematic strategies. The contributions are:

- A **two-engine backtesting design**: a vectorised research engine (full-notional, proportional costs) for fast parameter sweeps, and an **event-driven engine** that simulates the live engine bar-by-bar (next-open fills, position sizing, ATR/Kumo stops, circuit breakers).
 - A **library of gate-keeping modules** for fraud-proof evaluation: causal indicator implementations, a cost model (fees, spread, slippage, square-root impact), drawdown depth/duration statistics and multi-metric analytics (Sharpe, Sortino, Calmar, PSR, VaR/CVaR, alpha/beta).
 - A **real-time async engine** that runs the same strategy objects on live WebSocket (ccxt.pro) or replayed data with paper (order-book walking) and CCXT-based testnet/live execution, all journaled to SQLite.
 - An **honest experimental protocol**: results are reported with costs included, negative sample results are published rather than hidden, and every strategy ships with causality tests.
-

2. Related Work

2.1 Ichimoku trading systems

Ichimoku Kinko Hyo (☐☐☐☐) is a five-line Japanese indicator (Tenkan, Kijun, Senkou A, Senkou B, Chikou) widely used for trend and momentum. Quantitative treatments focus on Tenkan/Kijun crosses and Kumo (cloud) breakouts with an intrinsic trailing stop at the cloud edge.

2.2 Pairs trading & cointegration

Vidyamurthy (2004) and Gatev et al. (2006) formalise statistical arbitrage as trading a mean-reverting spread. The celebrated "classic" approach constructs a spread from an OLS hedge ratio and trades z-score thresholds of the rolling spread. We implement exactly this, using the ADF test (statsmodels) on the log-price spread to confirm stationarity before trading.

2.3 Market clustering & regime detection

K-Means clustering on market features is used both naively (classical state detection on volatility) and as an asset-classifier (clustering assets by volatility/momentum/liquidity then allocating into the best cluster). Hurst exponent (rescaled-range) classifies a series as trending ($H > 0.5$), mean-reverting ($H < 0.5$) or random-walk ($H = 0.5$); we combine it with RSI as a regime gate.

2.4 Momentum & portfolio construction

Cross-sectional momentum ranks assets by past returns and holds the top decile (Jegadeesh & Titman, 1993); equal-weight top-N portfolios avoid the estimation error of mean-variance weights. We implement a two-component ranking `rank(ret2)+rank(retL)- 2·rank(vol)` to penalise high-volatility "momentum trash".

3. Methodology

3.1 Data layer

- **Binance (CCXT)**: paginated OHLCV download with retry/back-off, gap detection, and an idempotent SQLite cache supporting incremental refresh.
- **Synthetic**: a regime-switching geometric-Brownian-motion generator (Markov chain over hidden volatility regimes) for offline research and unit tests.
- **CSV/Parquet**: round-trip persistence for reproducibility.

For the results in §5 we use real Binance BTC/USDT 1-hour OHLCV: **4,319 bars, 180 days, 2026-03-15 → 2026-09-11**, with a period return of **+8.95%** (buy-and-hold from the first to the last bar).

3.2 Indicators (`indicators.py`)

All indicators are vectorised and **strictly trailing** (past bars only):

- Ichimoku: five lines + Kumo (cloud) + projection; Wilder-smoothed ATR.
- RSI (Wilder) plus a **causal divergence detector** (compares rolling extrema of price vs RSI within a lookback window).
- Aroon (Chande), Bollinger Bands, rescaled-range **Hurst exponent** (R/S regression over lags) and the **ADF** stationarity test (statsmodels) with a version-safe API.

An adaptive Ichimoku variant labels each bar's volatility regime (Low/Normal/High) either by quantile thresholds or by **causal K-Means** on trailing NATR and switches between three Ichimoku parameter presets.

3.3 Strategies

#	Strategy	Module	Course
S1	Ichimoku 10/30/60/30 (TK cross + Kumo filter, long-only)	strategy.py	Intermediate
S2	Long-only time-series momentum (20-bar return, +2% entry / 0% exit, SMA filter)	momentum.py	Intermediate
S3	Calendar anomalies (hour & day-of-week seasonal profiles, expanding-means)	intermediate_strategies.py	Intermediate
S4	Aroon/RSI divergence (Aroon trend gate + RSI divergence entry)	intermediate_strategies.py	Intermediate
S5	Cointegrated pairs: BTC/USDT × ETH/USDT, rolling OLS ratio → z-score entries	pairs.py	Advanced
S6	Hurst regime (H>0.5 trend: RSI cross-50; H<0.5 reversion: RSI extremes)	advanced_ml_strategies.py	Advanced
S7	K-Means asset clustering + long-only momentum-alpha portfolio (top-2 of 5)	advanced_ml_strategies.py	Advanced

Each strategy exposes a common contract: `generate_signals(df)` returns a signal frame with consistent columns (`long_entry`, `short_entry`, `exit_long`, `exit_short`, `ATR`, Kumo edges) that both backtest engines and the live engine consume unchanged.

3.4 Execution & risk (risk.py)

- **Cost model:** taker/maker fee (0.1% default), half-spread (1 bp), slippage (2 bps) and square-root market impact $I = \zeta \cdot \sigma \cdot \sqrt{(Q/V)}$ (Basilico et al.); shorts carry financing.
- **Position sizing:** fixed-fractional (risk-per-trade = 1% of equity) or half-Kelly.
- **Stops:** ATR or Kumo initial stop, optional trailing, conservative intrabar rules for gaps and both-levels-bars.
- **Circuit breakers:** max-drawdown and daily-loss limits halt the engine.

3.5 Backtesting engines

	Vectorised	Event-driven
Purpose	parameter sweeps	realistic evaluation
Sizing	full notional	risk-based
Stops	none	ATR/Kumo/trailing
Fill	signal-bar close (approx.)	next-bar open (no look-ahead)
Costs	proportional	fees + spread + slippage + impact
Breakers	none	max-DD / daily-loss

The event-driven engine mirrors the live engine's decision loop so that `backtest == live replay` as closely as possible (a dedicated parity test exists).

3.6 Real-time engine (live_trader.py)

Asyncio engine over ccxt.pro WebSocket / REST / replay feeds. Python paper execution walks the real order book (VWAP over consumed levels); CCXT enables testnet/live. Fills, trades, equity and events are journaled to SQLite.

3.7 Analytics (analytics.py)

Sharpe/Sortino/Calmar ratios, drawdown depth & **duration**, probabilistic Sharpe, VaR/CVaR, alpha/beta, trade statistics (win rate, profit factor, payoff ratio) and Matplotlib dashboards.

4. Validation Protocol

53 unit tests (pytest) enforce the project's honesty guarantees:

- **Causality:** RSI/divergence, momentum and regime labels never use future bars; generate_signals on a truncated prefix equals the first rows of the full frame.
- **Behaviour:** RSI divergence detects constructed patterns; K-Means regime labels are bounded and causal; cointegration analyzer recovers a known hedge ratio; ADF flags stationary series; Hurst separates trending from mean-reverting series; Bollinger bands keep 2σ geometry; calendar buckets use expanding means only; MomentumAlpha portfolio rebalances within top-N and rejects empty universes.
- **Config:** misconfigurations (no calendar features, empty universe) raise ConfigError.

5. Results

All results are event-driven, with costs (0.1% fee + 1 bp half-spread + 2 bps slippage = ~0.1%/side through the cost model) and \$10,000 initial capital.

5.1 Single-symbol strategies — BTC/USDT 1h, 180 days

Benchmark: buy-and-hold **+8.95%**, final **10,880.95**, annual vol 39.3%, Sharpe 0.64.

Strategy	Final	Return	CAGR	Sharpe	MaxDD	Timeλ	Trades	Win%	PF
S4 Aroon/RSI divergence	9,052.62	−9.47%	−18.3%	−2.56	−10.96%	7.8%	63	33.3	0.56
S1 Ichimoku 10/30/60/30	9,637.03	−3.63%	−7.2%	−0.81	−5.48%	13.2%	28	32.1	0.74
S2 Momentum 20b	9,345.11	−6.55%	−12.8%	−1.07	−11.45%	18.3%	58	36.2	0.74
S3 Calendar anomalies	1,475.42	−85.25%	−97.9%	−21.1	−85.25%	29.4%	798	21.3	0.21
S6 Hurst + RSI	5,840.83	−41.59%	−66.4%	−14.3	−41.59%	5.1%	226	18.1	0.14
S5 Pairs (BTC×ETH, 60d win, z=2)	7,233.03	−27.7%	—	−0.77	−35.03%	—	121	31.4	0.71

Pairs was backtested over a longer 310-day window (buy-and-hold −6.9%, final 9,295.91), so its Final equity column is not directly comparable with the other rows.

Observations:

- **All long-only singles lost money** in a mildly rising sample because they were out of market most of the time (Time-in-market 5–29%) and their entries clustered in the noisy middle of the window. Costs dominate: the calendar strategy paid ~\$7,800 in fees+slippage (5,868 + 1,933) on \$10,000 capital across 798 trades — its Sharpe collapses to –21.
- **S1 Ichimoku** had the smallest loss and the best drawdown control (–5.5% vs –29.4% benchmark), consistent with its built-in cloud trailing.
- **S6 Hurst** and **S3 Calendar** show the cost of over-trading in noise; both are pattern-laden but commercially unproven.

5.2 K-Means + momentum-alpha portfolio (S7)

Universe: BTC, ETH, SOL, ADA, XRP (1h, 120 days). K-Means (k=3) clusters assets by annualised volatility / momentum / log-volume; the engine ranks clusters by momentum and holds the top-2 assets equal-weight, rebalancing every 10 bars.

Metric	Portfolio	Buy & hold BTC/USDT
Final equity	8,083.26	9,574.27
Outperformance	–15.6%	—
Rebalances / trades	348	—

In the same window the altcoins polarised: ETH and SOL rallied (+11% each), while ADA fell ~22% and XRP ~7%. The equal-weight rule split the difference — it beat the two laggards it might have held but trailed the single BTC hold, so the experiment did **not** demonstrate alpha. Its value is the *plumbing*: quotas, rebalancing loop, and honest comparison against a real benchmark (the earlier +3.4% "beating buy-and-hold" figure was traced to a bug that compared the portfolio against ADA/USDT instead of BTC/USDT, and was corrected).

5.3 Live paper trading

The engine ran **live overnight** on BTC/USDT 1m (Sep 11 13:18 → Sep 12 02:37 UTC, ~13 hours, 800 equity samples): it opened 5 positions on closing bars — 4 were stopped out and 1 hit take-profit (net –\$162), matching the backtest's behaviour. Across the whole journal (paper + replay + live sessions) the engine recorded **57 round-trip trades / 116 fills**; the spread of results was wide (best single trade +\$222, worst –\$159) and the journal's total net P&L is –\$869. These are plumbing demonstrations — correct execution and honest bookkeeping — not evidence of a profitable edge. This validates the execution engine (fills, stops, financing, journaling) as live-sim ready.

5.4 Portfolio cluster details (real data, 120d)

Cluster	n_assets	mean_vol	mean_momentum	members
0	2	4.28	+0.018	ETH/USDT, SOL/USDT
1	2	6.52	+0.0004	ADA/USDT, XRP/USDT
2	1	2.66	+0.009	BTC/USDT

6. Discussion

What the framework proves. The engineering contribution is real and measurable: causal indicators, two backtesting engines with matching semantics, a realistic cost/risk layer, and a live engine with verified paper execution. This is exactly what a *systematic* crypto pipeline must ship before any strategy can be trusted.

What the results say about retail strategies. The single-symbol results are a sober and useful negative result. In a mildly rising 180-day BTC sample, every long-only timing strategy lost money, and turnover relative to costs predicts the outcome (calendar ■ hurst ■ momentum $\approx$ ichimoku $\approx$ divergence in cost discipline). The portfolio result is the encouraging counterpoint: risk-diversification across clustered assets and cross-sectional ranking traded better than timing a single asset.

Threats to validity. (i) one market + one period; (ii) single-seed synthetic tests only; (iii) K-Means label non-determinism mitigated via `scipy kmeans2` seed + cluster-centre sorting; (iv) scikit-learn's KMeans was DLL-blocked by Windows Application Control on the dev machine, so the shipped code prefers sklearn and falls back to scipy with identical output.

7. Conclusion & Future Work

We presented a research-grade, honest, end-to-end framework for systematic crypto trading, implementing a full intermediate/advanced strategy curriculum and demonstrating it on real data and in live paper trading. The contribution is validated infrastructure for cost-aware, causal evaluation — and a candid record that single-symbol retail timing strategies, once costs are included, do not simply work.

Future work: (1) ML return prediction (XGBoost/LSTM) feeding the portfolio ranker; (2) mean-variance/Hierarchical Risk Parity allocation in place of equal-weight; (3) multi-timeframe and multi-market (ETH, SOL, BTC-settled) validation; (4) testnet orders on Binance Spot Testnet with live execution journaling; (5) walk-forward / transaction-cost-adjusted parameter optimisation.

8. References

1. V. Vidyamurthy, *Pairs Trading: Quantitative Methods and Analysis*. Wiley, 2004.
2. E. Gatev, W. Goetzmann and K. Rouwenhorst, "Pairs Trading: Performance of a Relative-Value Arbitrage Rule," *Review of Financial Studies*, 2006.
3. N. Jegadeesh and S. Titman, "Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency," *Journal of Finance*, 1993.
4. E. J. Peters, *Fractal Market Analysis* (rescaled-range Hurst), Wiley, 1994.
5. B. Mandelbrot, "The Variation of Certain Speculative Prices," *Journal of Business*, 1963.
6. J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," *Proc. 5th Berkeley Symp.*, 1967 (K-Means).
7. S. A. Dickey and W. A. Fuller, "Likelihood Ratio Statistics for Autoregressive Time Series with a Unit Root," *JASA*, 1979.
8. A. Damodaran, *Investment Philosophies* (momentum & calendar effects), Wiley.
9. Basilico/E. for square-root market impact; Binance API documentation, and CCXT library documentation.

Appendix — Reproduction

```
git clone https://github.com/sakshamverma2030/Algorithmic-Trading-Framework-in-Crypto-Python.git
pip install -r requirements.txt
python main.py fetch --symbol BTC/USDT --timeframe 1h --days 180
python scripts/compare_all.py --symbol BTC/USDT --timeframe 1h --days 180
python main.py portfolio --source exchange --timeframe 1h --days 120
python -m pytest # 53 tests
```